

POSIX Threads: Mutexes and Condition Variables

PGSCE – Programação Avançada de Sistemas Embebidos

António Barros

POSIX Threads: Mutexes

Shared Resources in POSIX Threads

Unlike separate processes, threads within the same process share:

- global variables and static data,
- heap memory (dynamically allocated with `malloc`), and
- file descriptors and process-wide signals.

However, simultaneous access to shared memory may lead to non-deterministic behaviour if not properly **synchronised**.

The Race Condition: Global Counter Example

Consider a program in which two threads increment a shared counter 1 000 000 times each.

```
volatile int counter = 0;

void* increment_counter(void* arg) {
    for (int i = 0; i < 1000000; i++) {
        counter++; // Critical section
    }

    pthread_exit(NULL);
}
```

```
int main()
{
    pthread_t tid[2];
    for(int i = 0; i < 2; i++) {
        pthread_create(&tid[i], NULL, increment_counter, NULL);
    }
    for(int i = 0; i < 2; i++) {
        pthread_join(tid[i], NULL);
    }
    printf("Shared counter: %d\n", counter);
    return 0;
}
```

The Race Condition: Global Counter Example

The expected result should be: $2 \times 1\,000\,000 = 2\,000\,000$.

Right?

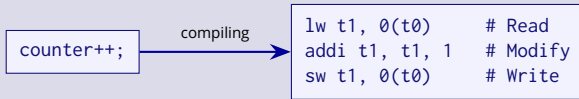
Actually, this result is **incorrect**: it is frequently less than 2 000 000.

```
user@machine PRASIE % ./race_condition
Shared counter: 1051952
user@machine PRASIE % ./race_condition
Shared counter: 1015934
user@machine PRASIE % ./race_condition
Shared counter: 1264826
user@machine PRASIE %
```

Dissecting counter++

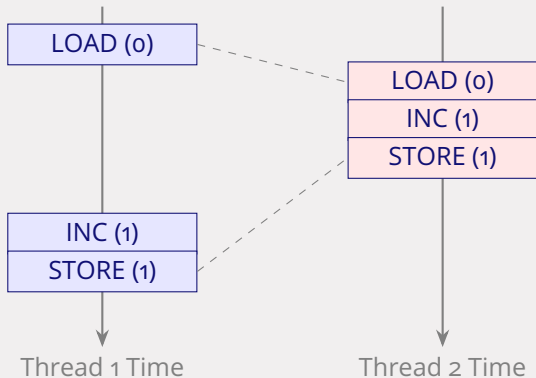
The high-level operation `counter++` is decomposed into 3 atomic CPU instructions:

1. **LOAD:** Read the value from memory into a CPU register.
2. **MODIFY:** Increment the value in the register.
3. **STORE:** Write the register value back to memory.



A **Race Condition** occurs when threads interleave during these steps. The final value depends on the unpredictable order of execution: the "*race*".

Visualising the Interleaving (Lost Update)



Result is 1, but it should have been 2.

The Critical Section

A **Critical Section** is a segment of code that accesses a shared resource (data, file, etc.) and must not be concurrently executed by more than one thread.

To prevent race conditions, we must ensure **Mutual Exclusion**.

Mutual exclusion is a concurrency control property that ensures exclusive access to shared resources for threads competing for that resource.

POSIX Solution: Mutex Locks

A **Mutex** (MUTual EXclusion) provides a way to protect critical sections.

- `pthread_mutex_lock()`: Acquire the lock if available; otherwise, block execution.
- `pthread_mutex_unlock()`: Release the lock to allow access by other threads.

The code becomes *Thread-Safe*:

```
pthread_mutex_lock(&lock_counter); // Acquire the mutex lock

// Critical section
counter++;

pthread_mutex_unlock(&lock_counter); // Release the mutex lock
```

Mutex Creation: `pthread_mutex_init()`

The standard way to initialise a mutex dynamically:

```
#include <pthread.h>

int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *attr);
```

- **Parameters:**
 - ▶ **mutex:** Pointer to the mutex object to be initialised.
 - ▶ **attr:** Pointer to attributes (use NULL for defaults).
- **Return Value:** 0 on success, or an error number.

Static Initialisation

For mutexes with default attributes that are *statically allocated*, a macro can be used.

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
```

- **Efficiency:** No function call overhead at runtime.
- **Simplicity:** No need to call `pthread_mutex_destroy()`.
- **Restriction:** Only applies to default mutex attributes.

Mutex Destruction: pthread_mutex_destroy()

The standard way to clean up resources allocated for a mutex:

```
#include <pthread.h>

int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

- **Parameters:**
 - ▶ **mutex:** Pointer to the mutex object to be destroyed.
- **Return Value:** 0 on success, or an error number (e.g. EBUSY).
- **Note:** A mutex must be **unlocked** before destruction to avoid undefined behaviour.

Locking & Unlocking: lock() and unlock()

The primary functions used to manage access to a critical section:

```
#include <pthread.h>
```

```
int pthread_mutex_lock(pthread_mutex_t *mutex);  
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

- **pthread_mutex_lock():**
 - ▶ Acquires the mutex. If the mutex is already locked, the calling thread **blocks** until it becomes available.
- **pthread_mutex_unlock():**
 - ▶ Releases the mutex. If other threads are blocked waiting for it, one of them will be woken up to acquire the lock.
- **Return Value:** 0 on success, or an error number.

Fixing the Race Condition: Synchronised Counter (1/2)

```
#include <stdio.h>
#include <pthread.h>

volatile int counter = 0;
pthread_mutex_t mtx_counter = PTHREAD_MUTEX_INITIALIZER;    // Static mutex

void* increment_counter(void* arg) {
    for (int i = 0; i < 1000000; i++) {
        pthread_mutex_lock(&mtx_counter);    // <-- Synchronise access to the critical section.

        counter++;

        pthread_mutex_unlock(&mtx_counter);    // <-- Release the mutex.
    }
    pthread_exit(NULL);
}
```

Fixing the Race Condition: Synchronised Counter (2/2)

```
int main()
{
    pthread_t tid[2];

    for(int i = 0; i < 2; i++) {
        pthread_create(&tid[i], NULL, increment_counter, NULL);
    }
    for(int i = 0; i < 2; i++) {
        pthread_join(tid[i], NULL);
    }
    printf("Shared counter: %d\n", counter);
    return 0;
}
```

Verification: Corrected Counter Output

```
user@machine PRASIE % ./race_condition_fixed
Shared counter: 2000000
user@machine PRASIE % ./race_condition_fixed
Shared counter: 2000000
user@machine PRASIE % ./race_condition_fixed
Shared counter: 2000000
user@machine PRASIE %
```


Best Practices: Mutex Usage

To ensure robust and performant concurrent programs, observe these fundamental principles when working with POSIX mutexes:

- **Resource-to-Mutex Mapping:**

- ▶ Explicitly define which shared resource (e.g., a specific variable or data structure) each mutex protects.
- ▶ Maintain a 1:1 conceptual relationship to avoid ambiguity.

- **Minimise Critical Section Duration:**

- ▶ Keep critical sections as short as possible to maximise parallel execution.
- ▶ Move any non-shared work outside the lock/unlock boundaries.

Best Practices: Safety and Determinism

Avoid Non-Deterministic Code

- **No Blocking Calls:** Never perform blocking operations (like `read()` or `sleep()`) while holding a mutex unless strictly necessary for safety.
- **Predictable Logic:** Ensure that the execution path within the critical section is finite and deterministic to prevent stalling the entire application.
- **Safe Exit Paths:** Ensure all return paths, breaks, or error handlers release the mutex. Use RAII-like patterns where possible.

Best Practices: Critical Section Integrity

A critical section must be strictly limited to the necessary synchronisation logic.

Avoid Non-Deterministic Code

Never place code within a critical section that:

- Could lead to an infinite loop or hang.
- Involves heavy blocking I/O (e.g., waiting for user input).
- Could trigger an exception or state that prevents reaching the `pthread_mutex_unlock()` call.

Any code that might fail to release the mutex will result in a **deadlock**, causing other threads to block indefinitely.

Condition Variables

Limitations of Mutexes

- Mutexes are designed for mutual exclusion, ensuring that only one thread accesses a resource at any given time.
- However, mutexes do not inherently signal when a condition changes:
 - ▶ E.g., is the buffer ready to be read?
 - ▶ Is the buffer available for writing?
- Must we resort to polling to check if a condition is met?

Example: Producer-Consumer with a Single-Slot Buffer

- Consider a scenario where a shared buffer holds exactly one element.
- **Synchronisation Requirements:**
 - ▶ **Producer:** Waits for the buffer to be empty; signals once an item has been stored.
 - ▶ **Consumer:** Waits for the buffer to be full; signals once an item has been consumed.

Example: Producer-Consumer with Single-Slot Buffer

Producer Logic:

```
pthread_mutex_lock(&mtx);

while (!(buffer_full == NO)) {
    pthread_mutex_unlock(&mtx);
    pthread_mutex_lock(&mtx);
}

// Write to buffer.
buffer_full = YES;

pthread_mutex_unlock(&mtx);
```

Consumer Logic:

```
pthread_mutex_lock(&mtx);

while (!(buffer_full == YES)) {
    pthread_mutex_unlock(&mtx);
    pthread_mutex_lock(&mtx);
}

// Read from buffer.
buffer_full = NO;

pthread_mutex_unlock(&mtx);
```

Continuous polling wastes CPU cycles and induces significant lock contention by repeatedly acquiring and releasing the mutex.

Addressing Busy Waiting

Condition Variables provide an efficient, event-driven synchronisation mechanism for threads.

- **Atomicity of Wait**

- ▶ When waiting upon a condition, a thread atomically releases the associated mutex and enters a suspended state.

- **Resumption and Re-acquisition**

- ▶ Upon receiving a signal, the thread resumes execution and automatically re-acquires the associated mutex before continuing.

The Wait-Signal Lifecycle

1. **Evaluate:** Thread locks mutex and checks the predicate.
2. **Block:** Thread calls wait; mutex is atomically released.
3. **Notify:** Another thread updates the shared state and signals the condition variable.
4. **Reacquire:** Thread wakes up and re-acquires the mutex lock.

Polling vs Efficient Sleeping

Polling Behaviour: Thread consumes CPU cycles while performing no useful work.

```
pthread_mutex_lock(&mtx);

while (!ready) {
    pthread_mutex_unlock(&mtx);
    pthread_mutex_lock(&mtx);
}

// Finally do work here.

pthread_mutex_unlock(&mtx);
```

Condition Variables: The thread is removed from the scheduler (blocked state) and releases the mutex.

```
pthread_mutex_lock(&mtx);

while (!ready) {
    pthread_cond_wait(&cond_var, &mtx);
}

// Finally do work here.

pthread_mutex_unlock(&mtx);
```

Condition Variable Creation: `pthread_cond_init()`

The standard way to initialise a condition variable dynamically:

```
#include <pthread.h>
int pthread_cond_init(pthread_cond_t *cond, const pthread_condattr_t *attr);
```

- **Parameters:**
 - ▶ **cond:** Pointer to the condition variable object to be initialised.
 - ▶ **attr:** Pointer to attributes (use NULL for default attributes).
- **Return Value:** 0 on success, or an error number if an error occurs.

Static Initialisation

For condition variables with default attributes that are *statically allocated*, a macro can be used.

```
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
```

- **Efficiency:** No function call overhead at runtime.
- **Simplicity:** No need to call `pthread_cond_destroy()`.
- **Restriction:** Only applies to condition variables with default attributes.

Condition Variable Destruction: pthread_cond_destroy()

The standard way to clean up resources allocated for a condition variable:

```
#include <pthread.h>

int pthread_cond_destroy(pthread_cond_t *cond);
```

- **Parameters:**
 - ▶ **cond:** Pointer to the condition variable object to be destroyed.
- **Return Value:** 0 on success, or an error number (e.g. EBUSY).
- **Note:** A condition variable must have **no threads currently waiting** on it before destruction to avoid undefined behaviour.

Waiting for a Condition: pthread_cond_wait()

The primary function used to suspend a thread until a specific condition is met:

```
#include <pthread.h>

int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);
```

- **Functionality:**

- ▶ Atomically releases the mutex and blocks the calling thread until the condition cond is signalled.
- ▶ Upon return, the thread has automatically re-acquired the mutex.

- **Important:** The caller must hold the mutex before calling this function to avoid race conditions.
- **Return Value:** 0 on success, or an error number.

Signalling a Thread: pthread_cond_signal()

Function used to notify (at least) one thread waiting on a condition variable:

```
#include <pthread.h>

int pthread_cond_signal(pthread_cond_t *cond);
```

- Unblocks **at least one thread** waiting on the specified condition variable.
- **Note:** If no threads are waiting on the condition variable, pthread_cond_signal() has no effect and the signal is lost.
- **Return Value:** 0 on success, or an error number.

Signalling all Threads: pthread_cond_broadcast()

```
#include <pthread.h>
```

```
int pthread_cond_broadcast(pthread_cond_t *cond);
```

- Unblocks **all threads** currently waiting on the specified condition variable.
- Useful when multiple threads can proceed following a state change (e.g., a “barrier” or a resource becoming available to many).
- **Note:** If no threads are waiting on the condition variable, pthread_cond_broadcast() has no effect and the signal is lost.
- **Return Value:** 0 on success, or an error number.

Best Practices: The Wait Loop

- A thread may occasionally wake up without a signal (*spurious wakeup*) or a signal might be intercepted by another thread.
- **The Rule:** Always wrap `pthread_cond_wait()` in a while loop to re-check the condition:

```
pthread_mutex_lock(&mutex);

while (!condition_is_met) {
    pthread_cond_wait(&cond, &mutex);
}

// Perform work...

pthread_mutex_unlock(&mutex);
```

Producer-Consumer with Single-Slot Buffer

```
pthread_mutex_t mtx;  
pthread_cond_t can_produce, can_consume;
```

Producer

```
pthread_mutex_lock(&mtx);  
  
while (buffer_state == BUFFER_FULL) {  
    pthread_cond_wait(&can_produce, &mtx);  
}  
  
// Write to buffer.  
buffer_state = BUFFER_FULL;  
pthread_cond_signal(&can_consume);  
  
pthread_mutex_unlock(&mtx);
```

Consumer

```
pthread_mutex_lock(&mtx);  
  
while (buffer_state == BUFFER_EMPTY) {  
    pthread_cond_wait(&can_consume, &mtx);  
}  
  
// Read from buffer.  
buffer_state = BUFFER_EMPTY;  
pthread_cond_signal(&can_produce);  
  
pthread_mutex_unlock(&mtx);
```

Use Case: Event-Driven Sensor Processing

- **Scenario:** A temperature sensor triggers data acquisition.
- **Producer Thread:** Reads the sensor and places data into a shared buffer.
- **Consumer Thread:** Waits for data to be available to perform calculations.
- **Goal:** Minimise CPU usage by avoiding busy waiting.

Shared Sensor Data Structure

Defining the Sensor State

```
typedef enum {  
    DATA_EMPTY,  
    DATA_READY  
} status_t;  
  
typedef struct {  
    int value;  
    status_t status;  
} sensor_sample_t;  
  
// Shared variable: must be protected by a mutex  
sensor_sample_t temperature;
```

Producer Thread: Sensor Acquisition

Producer Logic

```
pthread_mutex_lock(&mutex_temperature);

// Update data and set state
temperature.value = read_sensor(TEMPERATURE_001);
temperature.status = DATA_READY;

// Signal that data is available for consumption
pthread_cond_signal(&data_available);

pthread_mutex_unlock(&mutex_temperature);
```

Consumer Thread: Event Processing

Consumer Logic

```
pthread_mutex_lock(&mutex_temperature);

// Wait while data is not ready (handles spurious wakeups)
while (temperature.status != DATA_READY) {
    pthread_cond_wait(&data_available, &mutex_temperature);
}

// Consume the data
process(temperature.value);
temperature.status = DATA_EMPTY;

pthread_mutex_unlock(&mutex_temperature);
```

Communication Sequence

- **Step 1:** Consumer locks mutex and checks condition.
- **Step 2:** Consumer enters `pthread_cond_wait` (releases mutex).
- **Step 3:** Producer locks mutex, updates data, and signals.
- **Step 4:** Consumer wakes, re-acquires mutex, and processes data.